

# Table of Contents

---

## Week 3 — Attributes, Methods & Object State

Understanding What Objects Know and What They Can Do

Lesson Overview

Hour Plan

Recap of Week 2 (5 min)

Part 1 — Attributes: What an Object Currently Knows (13 min)

Open with a question (2 min)

Definition

Real-World Attribute Examples

Starting State

Part 2 — Two Kinds of Information: What Stays vs What's Temporary (12 min)

The Café Barista Story

Three Kinds of Information — Side by Side

The Nametag Story — Why “My Own” Matters

Part 3 — Methods: What an Object Can Do (12 min)

Open with a question (2 min)

Definition

Real-World Method Examples — Query vs Command

Methods Often Need Input (Parameters)

Methods Often Give Back an Answer (Return Value)

Some Actions Involve Two Objects Working Together

Part 4 — Object State Changing Over Time (10 min)

Bringing Back Sokha's Smartphone

A Parallel Example — A Student's GPA Through a Semester

Discussion Prompt

Part 5 — Java as Illustration (6 min)

Reading the Code as Concepts

Test Checkpoint

Common Misconceptions

Extension Challenges

● Basic — Identify the Three Kinds of Information

● Intermediate — Extend Your Smartphone Timeline

## Advanced — Two Objects Working Together

Connecting the Dots — Part 1 Review

Lesson Summary

Homework

# Week 3 — Attributes, Methods & Object State

## Understanding What Objects Know and What They Can Do

**Date:** Wednesday, 17 June 2026 | **Time:** 6:30 PM – 8:30 PM | **Course:** Object-Oriented Programming Concepts

### Lesson Overview

| Item          | Detail                                                                                                                                                                                                                             |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Topic         | Attributes, Methods & Object State                                                                                                                                                                                                 |
| Date & Time   | Wednesday, 17 June 2026 · 6:30 PM – 8:30 PM                                                                                                                                                                                        |
| Duration      | 2 hour                                                                                                                                                                                                                             |
| Format        | Online (Google Meet) — Concept explanation + real-world discussion + light illustration                                                                                                                                            |
| Prerequisites | Week 1 (What is OOP), Week 2 (Classes & Objects, independence)                                                                                                                                                                     |
| Focus         | OOP concept understanding — Java used only as illustration                                                                                                                                                                         |
| Outcome       | Students can explain attributes and methods in plain words, distinguish persistent vs temporary information using real-world analogies, tell command methods from query methods, and trace how an object’s state changes over time |

### Hour Plan

|               |                                                                   |
|---------------|-------------------------------------------------------------------|
| 00:00 – 00:05 | Recap of Week 2 + Homework check                                  |
| 00:05 – 00:18 | Part 1 – Attributes: What an Object Currently Knows               |
| 00:18 – 00:30 | Part 2 – Two Kinds of Information: What Stays vs What's Temporary |
| 00:30 – 00:42 | Part 3 – Methods: What an Object Can Do                           |
| 00:42 – 00:52 | Part 4 – Object State Changing Over Time (real-world walkthrough) |
| 00:52 – 00:58 | Part 5 – Java as Illustration (light)                             |
| 00:58 – 01:00 | Checkpoint + Homework                                             |

 **Google Meet tip:** Keep *Week3\_Summary\_Examples.md* open in a second tab — it has the café barista and smartphone timeline examples ready to paste into chat if students need a recap mid-lesson.

## Recap of Week 2 (5 min)

Ask students verbally:

1. "What is the difference between a Class and an Object? Give me an example from your own life — not Student, not Smartphone."
2. "If Sokha's bank account balance changes, does Dara's bank account change too? Why or why not?"
3. "Quick fire: is 'Book' a class or an object? Is 'Clean Code' by Robert Martin, currently checked out by Dara' a class or an object?"
4. "From homework — you designed a Smartphone class with 3 objects: Sokha's, Dara's, and Chan's phones. If Sokha's battery drops to 5%, does that affect Dara's or Chan's phone? Why not?"

*Today we use your own Smartphone homework as the running example — keep it nearby.*

## Part 1 — Attributes: What an Object Currently Knows (13 min)

### Open with a question (2 min)

*"Right now, in this exact moment — what do you currently know about your own condition? Are you hungry? Tired? What is your mood? How much energy do you have left today?"*

Let students answer freely.

*"Everything you just described is your **current state** — information you hold about yourself, right now, at this exact moment. In OOP, we call this an object's **attributes**."*

### Definition

**Attributes** are the pieces of information an object holds about itself.

Together, all of an object's attributes form its **state** — a snapshot of what it currently knows about itself.

Attributes are not actions. They are not behaviors. They are simply: *information the object carries*.

## Real-World Attribute Examples

Work through these together. Ask “what would this object currently know about itself?” before revealing.

| OBJECT          | ATTRIBUTES (what it currently knows about itself) |
|-----------------|---------------------------------------------------|
| A person        | energyLevel, mood, hungerLevel, age               |
| A smartphone    | batteryPercent, volumeLevel, isOn, storageUsed    |
| A tuk-tuk       | fuelLevel, currentSpeed, numberOfPassengers       |
| A water bottle  | amountOfWaterInside, temperature                  |
| A bank account  | balance, accountType, transactionCount            |
| A student       | currentGPA, creditsCompleted, enrollmentStatus    |
| A traffic light | currentColor (red/yellow/green)                   |
| A rice cooker   | temperature, isOn, cookingMode                    |

**Reminder from Week 2:** Every object has its OWN copy of these attributes. Sokha's phone being at 20% battery has zero effect on Dara's phone. Each object's state is independent.

## Starting State

When something brand new is created, it begins with some starting condition.

### EXAMPLES OF STARTING STATE:

|                                        |                                         |
|----------------------------------------|-----------------------------------------|
| A new smartphone, fresh out of the box | → batteryPercent = 100                  |
| A new bank account, just opened        | → balance = 0 (or opening deposit)      |
| A new student, just enrolled           | → creditsCompleted = 0                  |
| A new water bottle, unused             | → amountOfWaterInside = 0 (empty)       |
| A new traffic light, just installed    | → currentColor = "red" (a safe default) |

*Every object begins somewhere — and from that starting point, its state changes over time through actions. That is exactly what Part 3 and Part 4 explore today.*

# Part 2 — Two Kinds of Information: What Stays vs What's Temporary (12 min)

## The Café Barista Story

"Picture a barista at a coffee shop. Throughout her whole shift, she keeps a running tally in her head: **'drinks made today.'** This number belongs to HER. It stays with her the entire shift. This is like a **persistent attribute** — it belongs to the object and lasts as long as the object exists."

"Now — a customer walks up and says: **'I'd like a medium iced coffee, less sugar.'**  
This instruction — medium, iced, less sugar — is given TO the barista, just for THIS ONE order. Once she makes the drink and hands it over, this specific instruction is forgotten — it doesn't stick around for the next customer. This is like a **parameter** — information given from outside, relevant only for one specific action."

"While making this one drink, the barista might think: **'I need exactly 200ml of milk for this cup.'**  
This number only matters for making this one drink, right now. Once the drink is made, that calculation is gone — never written down anywhere permanent. This is like a **local, temporary value** — created to help finish one action, then thrown away."

## Three Kinds of Information — Side by Side

|                           | PERSISTENT<br>(instance attribute)                      | GIVEN FROM OUTSIDE<br>(parameter)                                                | TEMPORARY HELPER<br>(local value)                           |
|---------------------------|---------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------|
| Belongs to ...<br>Example | The object, forever<br>"drinks made today"<br>(barista) | One specific action<br>customer's order details<br>(medium, iced, less<br>sugar) | One specific action<br>milk measurement<br>for this one cup |
| Lifespan                  | As long as the<br>object exists                         | Only during that one<br>action                                                   | Only during that one<br>action, then forgotten              |
| Real example 2            | Phone's batteryPercent                                  | "charge for 20 minutes"<br>(told to the phone)                                   | "battery gained = 10%"<br>(calculated mid-action)           |
| Real example 3            | Bank account balance                                    | "deposit \$200"<br>(told to the account)                                         | "fee = \$1 if over limit"<br>(calculated mid-action)        |

## The Nametag Story — Why “My Own” Matters

This addresses the single most common beginner mistake. Tell it as a story.

*“Imagine you’re at a party. Someone new walks up and says: ‘Hi, my name is Sokha.’*

*You want to remember this — so you take out YOUR OWN phone and save ‘Sokha’ into YOUR contacts.*

*But imagine instead, you just repeated the word ‘Sokha’ out loud and walked away — without writing it into your own contacts. The next day, you’d have completely forgotten the name — because you never actually saved it into YOUR OWN permanent memory. You just bounced the word back into the air and let it disappear.”*

### THE MISTAKE:

---

|                    |                                            |                            |
|--------------------|--------------------------------------------|----------------------------|
| Someone tells you: | "My name is Sokha"                         | (information given to you) |
| You say:           | "Sokha"                                    | (you just repeat it back)  |
| Result:            | Nothing was saved. You forget it tomorrow. |                            |

### THE FIX:

---

|                    |                                                                            |                            |
|--------------------|----------------------------------------------------------------------------|----------------------------|
| Someone tells you: | "My name is Sokha"                                                         | (information given to you) |
| You think:         | "Let me save THIS into MY OWN contacts"                                    |                            |
| You do:            | [Open MY contacts] → [Save: Sokha]                                         |                            |
| Result:            | Tomorrow, you still remember – it's saved<br>in YOUR OWN permanent record. |                            |

---

*“In OOP, when we update an object’s own permanent information using something given from outside, we must be clear: ‘**save this into MY OWN record**’ — not just repeat the word back. This idea of ‘my own’ is what the keyword `this` represents in code. It means: the object’s own permanent copy, as opposed to the temporary information someone just handed me.”*

## Part 3 — Methods: What an Object Can Do (12 min)

### Open with a question (2 min)

*“Think about your smartphone from last week’s homework. What actions can it perform?”*

Students will list: make a call, take a photo, charge, check battery, send a message, etc.

*"Everything you just listed is a **method** — an action the object can perform."*

## Definition

**Methods** are the actions (behaviors) an object can perform.

There are two basic flavors:

### QUERY METHOD

"just tells you something"

Reads/reports current state

Nothing about the object changes

Example: "What's my battery level?"

Example: "What's my balance?"

Example: "How many drinks made?"

### COMMAND METHOD

"actually changes something"

Changes the object's state

Something about the object changes

Example: "Charge the phone"

Example: "Deposit \$200"

Example: "Make one more drink"

## Real-World Method Examples — Query vs Command

| OBJECT        | QUERY (just reports) | COMMAND (actually changes) |
|---------------|----------------------|----------------------------|
| Smartphone    | checkBattery()       | charge(), makeCall()       |
| Bank account  | checkBalance()       | deposit(), withdraw()      |
| Tuk-tuk       | checkFuelLevel()     | refuel(), drive()          |
| Student       | checkGPA()           | submitExam()               |
| Water bottle  | checkWaterLevel()    | drinkSome(), refill()      |
| Traffic light | checkCurrentColor()  | changeToNext()             |

**Quick test:** "If I ask my phone 'what's my battery?' — does asking that question use up any battery itself? No — asking is a query. It only reports. Charging the phone, however, actually changes the battery level — that's a command."



## Methods Often Need Input (Parameters)

*"Some actions need information from outside to know exactly what to do.  
If I tell my phone '**call someone**' — it needs to know WHO. That's input.  
If I tell the bank '**deposit money**' — it needs to know HOW MUCH. That's input.  
This input is what we called a **parameter** in Part 2 — information given to the action, relevant only for that one action."*

## Methods Often Give Back an Answer (Return Value)

*"Some actions give you back a result. Asking '**what's my balance?**' gives you back a number.  
Asking a thermometer '**what's the temperature?**' gives you back a value.  
This is the **output** of an action — sometimes called a return value."*

## Some Actions Involve Two Objects Working Together

*"Some actions are not just about ONE object — they connect two objects.  
When Sokha sends money to Dara, this is ONE action ("**transfer**") — but it involves TWO bank accounts. Sokha's account decreases. Dara's account increases.  
Or think of a teacher grading a specific student's test — the teacher's action needs to know WHICH student, and it changes THAT student's record."*

### TWO-OBJECT INTERACTION EXAMPLES

|                                             |                                        |
|---------------------------------------------|----------------------------------------|
| <code>Sokha's BankAccount.transfer()</code> | → involves Dara's BankAccount too      |
| <code>Teacher.gradeTest()</code>            | → involves a specific Student object   |
| <code>Barista.serveOrder()</code>           | → involves a specific Customer's order |
| <code>Doctor.treatPatient()</code>          | → involves a specific Patient object   |

## Part 4 — Object State Changing Over Time (10 min)

### Bringing Back Sokha's Smartphone

*"Let's trace your own Week 2 homework. Remember Sokha's phone? Let's follow what happens to its state across one school day."*

#### SOKHA'S SMARTPHONE – ONE DAY TIMELINE

| Time     | Action (method called)   | State after the action               |
|----------|--------------------------|--------------------------------------|
| 7:00 AM  | Phone turned on          | batteryPercent = 100                 |
| 7:30 AM  | makeCall()               | batteryPercent = 95                  |
| 10:00 AM | takePhoto() ×10          | batteryPercent = 80, photoCount = 10 |
| 1:00 PM  | playGame() for 1 hour    | batteryPercent = 45                  |
| 3:00 PM  | charge()                 | batteryPercent = 100                 |
| 6:00 PM  | checkBattery() ← a QUERY | batteryPercent = 100 (unchanged!)    |

**Key teaching point:** “Every single change happened because of an **action** (a method call). The battery never changed by itself, randomly. State changes **ONLY** through actions.”

**Second key point:** “At 6:00 PM, Sokha just checked her battery — a query. Notice: nothing changed. Only command actions change state. Queries only report.”

## A Parallel Example — A Student's GPA Through a Semester

#### SOKHA THE STUDENT – ONE SEMESTER TIMELINE

| Week    | Action (method called)  | State after the action          |
|---------|-------------------------|---------------------------------|
| Week 1  | Enroll                  | gpa = 0.0, creditsCompleted = 0 |
| Week 8  | submitExam() – mid-term | gpa = 3.2                       |
| Week 15 | submitExam() – final    | gpa = 3.5                       |
| Anytime | checkGPA() ← a QUERY    | gpa unchanged                   |

“And remember — while Sokha's GPA is changing through her semester, Dara's GPA is doing its own completely separate thing. Neither student's record affects the other's. This is independence, from Week 2, showing up again here.”

## Discussion Prompt

*"If I told you a water bottle went from 'half full' to 'empty' — what ACTION must have happened? Could it have happened with no action at all?"*

*(Expected: someone must have drunk from it or poured it out — state never changes without an action.)*

## Part 5 — Java as Illustration (6 min)

**Reminder:** Java is just how we WRITE these concepts down. The concept came first — the code is only a translation.

```
// CLASS - defines what every Smartphone will know and do
public class Smartphone {

    // ATTRIBUTES - persistent state, belongs to each object
    String brand;
    int batteryPercent;
    boolean isOn;

    // COMMAND method - changes the object's state
    void makeCall() {
        int cost = 5; // a LOCAL value - used once here, then forgotten
        batteryPercent = batteryPercent - cost;
        System.out.println(brand + " made a call. Battery: " + batteryPercent + "%");
    }

    // COMMAND method WITH A PARAMETER - input given from outside
    void charge(int minutes) {
        // 'minutes' is a PARAMETER - given by whoever calls this action
        int gained = minutes / 2; // local - temporary helper value
        batteryPercent = batteryPercent + gained;
        if (batteryPercent > 100) batteryPercent = 100;
        System.out.println(brand + " charged " + minutes + " min. Battery: " + batteryPercent + "%");
    }

    // QUERY method - only reports state, never changes it
    int checkBattery() {
        return batteryPercent;
    }
}
```

```
// Using the Smartphone object – same as the timeline we traced above
Smartphone sokhaPhone = new Smartphone();
sokhaPhone.brand      = "Samsung";
sokhaPhone.batteryPercent = 100;

sokhaPhone.makeCall();           // battery drops – a command
sokhaPhone.makeCall();           // battery drops again
sokhaPhone.charge(20);           // battery rises – a command with input

System.out.println("Current battery: " + sokhaPhone.checkBattery()); // a query
```

## Reading the Code as Concepts

CONCEPT TRANSLATION TABLE

| Java code             | OOP concept meaning                             |
|-----------------------|-------------------------------------------------|
| brand, batteryPercent | Attributes – persistent state                   |
| minutes (in charge()) | Parameter – given from outside, for this action |
| cost, gained          | Local values – temporary, used once, forgotten  |
| makeCall(), charge()  | Command methods – they CHANGE state             |
| checkBattery()        | Query method – it only READS state              |
| sokhaPhone            | The object – a real instance with real data     |

## Test Checkpoint

- ☐ Can explain what an “attribute” is using a real-world example that is not a phone
- ☐ Can explain the difference between persistent, given-from-outside, and temporary information using the barista story
- ☐ Can explain why “my own” matters using the nametag/contact-book story
- ☐ Can tell a command method from a query method, with an example of each
- ☐ Can trace a simple real-world state timeline (e.g., predict a water bottle’s state across fill/drink actions) and explain that state never changes without an action

## Common Misconceptions

| Misconception                                                | Clarification                                                                                                                                             |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Attributes and methods are the same thing"                  | No — attributes are what the object <i>knows</i> (data); methods are what it <i>can do</i> (actions).                                                     |
| "If I just repeat information back, the object remembers it" | No — like the nametag story, information must be explicitly saved into the object's OWN permanent record, or it's forgotten.                              |
| "Checking something changes it"                              | No — query methods only report. Only command methods change state.                                                                                        |
| "An object's state changes by itself"                        | No — state only changes when an action (method) is performed on it. Nothing changes randomly.                                                             |
| "Temporary calculations get saved permanently"               | No — local, temporary values exist only during one action and disappear right after, unless their result is explicitly saved into a persistent attribute. |

## Extension Challenges

### ● Basic — Identify the Three Kinds of Information

Pick one real object from your daily life (a school bag, a motorbike, a rice cooker).

- List 4 attributes (its current state)
- List 3 methods: 2 commands and 1 query
- For one of your command methods, describe what "temporary helper information" it might briefly need while performing the action

### ● Intermediate — Extend Your Smartphone Timeline

Using the Smartphone you designed in Week 2 homework:

- Write a full-day timeline with **at least 6 events** (similar to the one we built in class)
- Show how `batteryPercent`, `storageUsed`, and `photoCount` change through different actions (calls, photos, charging, checking)
- Mark clearly which events are command methods and which are query methods

## ● Advanced — Two Objects Working Together

Design a scenario where TWO objects interact (similar to "Sokha transfers to Dara" or "Teacher grades Student"). You can use a `TukTukDriver` and a `Passenger`, or a `Barista` and a `Customer`.

In plain English, describe:

- What action (method) triggers the interaction?
- What input (parameter) does it need from outside?
- How does each object's state change as a result?
- Is this method a command or a query? How do you know?

---

## Connecting the Dots — Part 1 Review

### PART 1: FOUNDATIONS – WHAT WE HAVE LEARNED

---

#### WEEK 1 – What is OOP?

- OOP organizes code as interacting objects, not loose data and functions
- 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction

#### WEEK 2 – Classes & Objects

- Class = template/blueprint (no data); Object = real instance (real data)
- Every object holds its own independent copy of data
- Changing one object never affects another

#### WEEK 3 – Attributes, Methods & Object State

- Attributes = what an object currently knows about itself (its state)
- Methods = what an object can do (its behaviors)
- Persistent info (attribute) vs given-from-outside (parameter)  
vs temporary helper (local) – three different lifespans
- "My own" (this) – saving given information into the object's  
OWN permanent record, not just repeating it back
- Command methods change state; query methods only read it
- State changes ONLY through actions – never by itself

#### NEXT: Week 4 – Encapsulation

- We will learn how to PROTECT an object's state
  - So it cannot be changed carelessly or incorrectly from outside
  - We will see exactly why this protection matters
-

# Lesson Summary

## TODAY'S KEY TAKEAWAYS

1. Attributes are what an object currently KNOWS about itself.  
Together, they form the object's state.
2. Methods are what an object CAN DO – its actions/behaviors.
3. Three kinds of information, three different lifespans:  
Persistent (attribute) → lives with the object, forever  
Given from outside (parameter) → lives only for one action  
Temporary helper (local) → lives only during one action,  
then disappears, unless explicitly saved
4. "My own" (this) means: save information given to me into  
MY OWN permanent record – not just repeat it back.  
This is the nametag/contact-book lesson.
5. Command methods CHANGE an object's state.  
Query methods only READ and report it – nothing changes.
6. An object's state changes ONLY through actions.  
It never changes randomly or by itself.

## Homework

Extend the `Smartphone` design from your Week 2 homework — plain English/diagram is enough, no Java code required:

1. Add a new attribute: `storageUsedGB`
2. Add three methods:
  - `takePhoto()` — a **command**: increases `storageUsedGB`, decreases `batteryPercent` slightly
  - `deleteAllPhotos()` — a **command**: resets `storageUsedGB` back to 0
  - `checkStorage()` — a **query**: just reports the current `storageUsedGB`, changes nothing
3. Write a **6-event timeline table** (like the one we built in class) showing Sokha's phone state changing across one day — include at least one query event where nothing changes

**Optional bonus (only if you want extra practice):** try writing this as actual Java code, following today's illustration as a guide.

**Bring your timeline to Week 4** — we will use it to explore why some of this state needs to be *protected* from being changed incorrectly.

---

*Week 3 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com*